



电子科技大学
格拉斯哥海南学院
Glasgow College Hainan, UESTC

UESTC 4014: Real-Time Computing Systems and Architecture

Fall Semester 2026-2027

Lab 2 - Week 2

Report Due Date: Wednesday, 30th Sept 2026, 23:59 CST

Student's Name	Zhanyu Zhang
Student's UoG ID	2959456
Student's UESTC ID	2023300901013
Date	18 September 2026

Abstract

This laboratory implements UART, I2C, and SPI communication together with pulse-width modulation. A UART-driven calculator receives three two-digit integers and reports either their sum or product. The STM32 then operates as an I2C master to control an Arduino LED interval and read either a two-byte analog sample or a four-byte character sequence. The same analog sample is acquired through SPI and mapped first to an LED duty cycle and then to a servo command pulse. The four tasks combine terminal input, peripheral transactions, byte reconstruction, and timer compare control in a modular application.

1 Introduction

Embedded systems commonly combine several communication interfaces because each protocol serves a different purpose. UART provides a direct text interface, I2C connects addressed devices on a two-wire bus, and SPI offers synchronous full-duplex transfers controlled by chip select. PWM converts numerical control values into timed output waveforms suitable for LED brightness and servo position control.

Each task is implemented as a separate function called from the main loop. USART2 handles text input and output, while I2C1 and SPI1 communicate with the supplied Arduino slave programs. TIM2 and TIM21 generate the two PWM outputs.

2 Task 1: UART Calculator

USART2 is configured for asynchronous communication at 115200 bit/s with eight data bits, no parity, and one stop bit. The program reads each input line through UART before processing it.

The first line contains three space-separated two-digit integers. The next input selects addition or multiplication. Listing 1 shows the UART input and arithmetic functions. A 32-bit result represents the largest permitted product, $99^3 = 970299$, directly.

Listing 1: UART calculator input and arithmetic.

```
1  #include "task1.h"
2
3  #include <stdio.h>
4  #include <string.h>
5
6  static void ReadLine(char *line)
7  {
8      uint8_t byte;
9      uint8_t i = 0U;
10
11     while (1) {
12         HAL_UART_Receive(&huart2, &byte, 1U, HAL_MAX_DELAY);
13         if ((byte == '\r') || (byte == '\n')) {
14             line[i] = '\0';
15             return;
16         }
17         line[i++] = (char)byte;
18     }
19 }
20
21 static void Print(const char *text)
22 {
23     HAL_UART_Transmit(&huart2, (uint8_t *)text,
24                      (uint16_t)strlen(text), HAL_MAX_DELAY);
25 }
26
27 void Task1(void)
28 {
29     char line[32];
30     uint8_t values[3];
31     uint32_t result;
32
33     Print("Numbers: ");
34     ReadLine(line);
35     sscanf(line, "%hhu %hhu %hhu",
36            &values[0], &values[1], &values[2]);
```

```

37
38 Print("Operation (+/*): ");
39 ReadLine(line);
40
41 if (line[0] == '+') {
42     result = (uint32_t)values[0] + values[1] + values[2];
43 } else {
44     result = (uint32_t)values[0] * values[1] * values[2];
45 }
46
47 snprintf(line, sizeof(line), "Result: %lu\r\n", (unsigned long)result);
48 Print(line);
49 }

```

For the test data 12, 34, and 56, the two requested operations are

$$12 + 34 + 56 = 102, \quad 12 \times 34 \times 56 = 22848.$$

Figure 1 shows the input and result for each arithmetic operation.

```

Numbers: 12 34 56
Operation (+/*): +
Result: 102

```

```

Numbers: 12 34 56
Operation (+/*): *
Result: 22848

```

Figure 1: Calculator results displayed in the serial terminal.

3 Task 2: I2C Command and Data Selection

The STM32 is the I2C1 master and the Arduino is the slave at seven-bit address 0x55. The first UART selection is converted to one of the commands 0x80–0x83, which selects the Arduino LED interval. The second selection chooses register 0x00 or 0x01.

Register 0x00 returns a two-byte analog sample in little-endian order. If the received bytes are b_0 and b_1 , the value is reconstructed as

$$A = b_0 + (b_1 \ll 8).$$

Register 0x01 returns the four ASCII bytes RTCA. Listing 2 shows the command write, register selection, read transaction, and data reconstruction.

Listing 2: I2C command and data exchange.

```

1 #include "task2.h"
2
3 #include <stdio.h>
4 #include <string.h>
5
6 #define ARDUINO_ADDRESS (0x55U << 1U)
7
8 static void ReadLine(char *line)
9 {
10     uint8_t byte;
11     uint8_t i = 0U;
12
13     while (1) {

```

```

14 HAL_UART_Receive(&huart2, &byte, 1U, HAL_MAX_DELAY);
15 if ((byte == '\r') || (byte == '\n')) {
16     line[i] = '\0';
17     return;
18 }
19 line[i++] = (char)byte;
20 }
21 }
22
23 static void Print(const char *text)
24 {
25     HAL_UART_Transmit(&huart2, (uint8_t *)text,
26                     (uint16_t)strlen(text), HAL_MAX_DELAY);
27 }
28
29 void Task2(void)
30 {
31     char line[32];
32     char output[64];
33     uint8_t command;
34     uint8_t register_address;
35     uint8_t data[4];
36     unsigned int speed;
37     unsigned int select_data;
38
39     Print("Speed (0-3), data (0/1): ");
40     ReadLine(line);
41     sscanf(line, "%u %u", &speed, &select_data);
42
43     command = 0x80U + (uint8_t)speed;
44     register_address = (uint8_t)select_data;
45     HAL_I2C_Master_Transmit(&hi2c1, ARDUINO_ADDRESS,
46                             &command, 1U, HAL_MAX_DELAY);
47     HAL_I2C_Master_Transmit(&hi2c1, ARDUINO_ADDRESS,
48                             &register_address, 1U, HAL_MAX_DELAY);
49
50     if (select_data == 0U) {
51         HAL_I2C_Master_Receive(&hi2c1, ARDUINO_ADDRESS,
52                                data, 2U, HAL_MAX_DELAY);
53         snprintf(output, sizeof(output), "0x00: %u\r\n",
54                 (unsigned int)(data[0] | (data[1] << 8U)));
55     } else {
56         HAL_I2C_Master_Receive(&hi2c1, ARDUINO_ADDRESS,
57                                data, 4U, HAL_MAX_DELAY);
58         snprintf(output, sizeof(output), "0x01: %c%c%c%c\r\n",
59                 data[0], data[1], data[2], data[3]);
60     }
61
62     Print(output);
63 }

```

I2C transaction sequence

Stage	Transferred value	Purpose
Write 1	0x80–0x83	Select LED interval
Write 2	0x00 or 0x01	Select slave data
Read	2 or 4 bytes	Acquire analog sample or text

Figure 2 shows speed selection 2, which sends 0x82. Data selection 0 reads register 0x00; data selection 1 reads register 0x01.

```

Speed (0-3), data (0/1): 2 0
0x00: 673

Speed (0-3), data (0/1): 2 1
0x01: RTCA

```

Figure 2: Analog and text values displayed in the serial terminal.

4 Task 3: SPI Analog Input to LED PWM

SPI1 operates as a mode-0, MSB-first master with a prescaler of 128. The 32 MHz peripheral clock therefore produces

$$f_{\text{SCK}} = \frac{32 \text{ MHz}}{128} = 250 \text{ kHz}.$$

The Arduino exposes the lower and upper ADC bytes through commands 0x00 and 0x01. Each register read sends its command and then a dummy byte to generate the clocks for the reply. The two bytes are combined as a 16-bit value.

TIM2 generates the LED waveform. With PSC = 31 and ARR = 999,

$$f_{\text{PWM}} = \frac{32\,000\,000}{(31 + 1)(999 + 1)} = 1 \text{ kHz}.$$

For a 10-bit Arduino sample $A \in [0, 1023]$, the compare value is

$$CCR_{\text{LED}} = \left\lfloor \frac{1000A}{1023} \right\rfloor, \quad D = 100 \frac{CCR_{\text{LED}}}{1000} \%.$$

Thus $A = 0$ produces 0%, $A = 512$ produces approximately 50%, and $A = 1023$ produces 100% duty cycle. The SPI transaction and LED mapping are shown together in Listing 3.

5 Task 4: SPI Analog Input to Servo PWM

TIM21 generates the servo control waveform. The same prescaler gives a 1 MHz counter, so one timer count represents 1 μs . With ARR = 19999, the PWM period is

$$T_{\text{servo}} = \frac{(31 + 1)(19999 + 1)}{32\,000\,000} = 20 \text{ ms}, \quad f_{\text{servo}} = 50 \text{ Hz}.$$

The analog input is mapped linearly to a pulse width from 1000 μs to 2000 μs :

$$CCR_{\text{servo}} = 1000 + \left\lfloor \frac{1000A}{1023} \right\rfloor.$$

Under a nominal 0°–180° servo mapping, the corresponding command angle is

$$\theta = \left\lfloor \frac{180A}{1023} \right\rfloor.$$

The end points therefore command 1000 μs at 0° and 2000 μs at 180°, while the midpoint is approximately 1500 μs at 90°.

Listing 3: SPI analog acquisition and PWM mappings for Tasks 3 and 4.

```

1  #include "task34.h"
2
3  #include <stdio.h>
4  #include <string.h>
5
6  static void Print(const char *text)
7  {
8      HAL_UART_Transmit(&huart2, (uint8_t *)text,
9                          (uint16_t)strlen(text), HAL_MAX_DELAY);
10 }
11
12 static uint8_t ReadByte(uint8_t command)
13 {
14     uint8_t empty;
15     uint8_t value;
16     uint8_t dummy = 0xFFU;
17
18     HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_RESET);
19     HAL_SPI_TransmitReceive(&hspi1, &command, &empty, 1U, HAL_MAX_DELAY);
20     HAL_Delay(1U);
21     HAL_SPI_TransmitReceive(&hspi1, &dummy, &value, 1U, HAL_MAX_DELAY);
22     HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_SET);
23
24     return value;
25 }
26
27 static uint16_t ReadAnalog(void)
28 {
29     uint16_t low = ReadByte(0x00U);
30     uint16_t high = ReadByte(0x01U);
31
32     return low | (high << 8U);
33 }
34
35 void Task3(void)
36 {
37     uint16_t analog_value = ReadAnalog();
38     uint32_t compare = (analog_value * 1000U) / 1023U;
39     char output[64];
40
41     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, compare);
42     snprintf(output, sizeof(output), "ADC: %u, LED duty: %lu%%\r\n",
43              analog_value, (unsigned long)(compare / 10U));
44     Print(output);
45 }
46
47 void Task4(void)
48 {
49     uint16_t analog_value = ReadAnalog();
50     uint32_t pulse = 1000U + (analog_value * 1000U) / 1023U;
51     char output[64];
52
53     __HAL_TIM_SET_COMPARE(&htim21, TIM_CHANNEL_1, pulse);
54     snprintf(output, sizeof(output), "ADC: %u, servo pulse: %lu us\r\n",
55              analog_value, (unsigned long)pulse);
56     Print(output);
57 }

```

Figure 3 shows both transfer functions. The LED duty cycle and servo pulse width increase monotonically across the complete 10-bit input range.

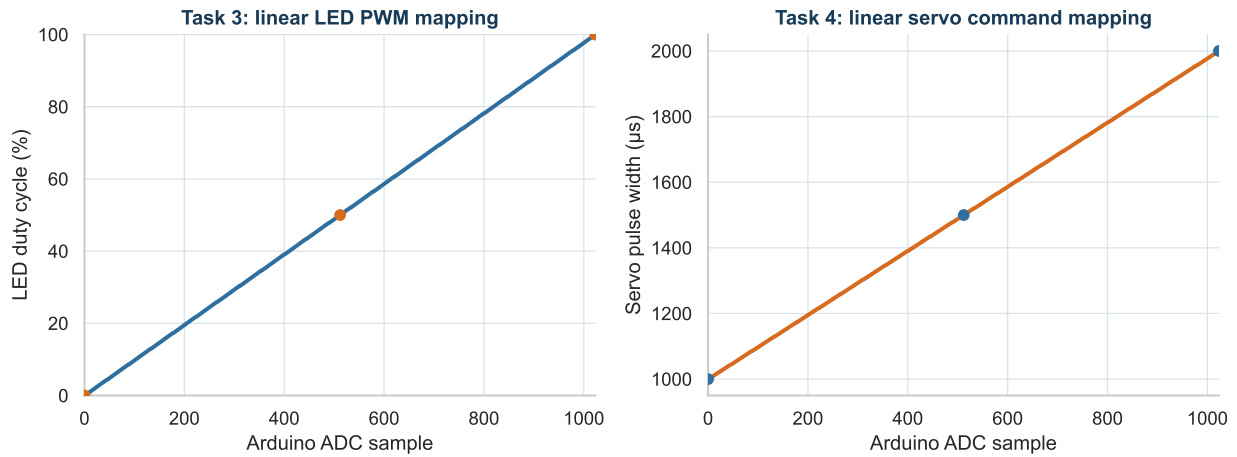


Figure 3: ADC-to-PWM mappings used for LED brightness and servo position.

6 Conclusion

The laboratory integrated three serial interfaces with two timer-controlled outputs. UART input supported the calculator and parameter entry. I2C transactions selected the Arduino LED interval and retrieved either a reconstructed analog value or the four-byte text response. SPI register transactions acquired the same 10-bit sample as two bytes. Linear timer mappings then converted this sample into a 1 kHz LED duty cycle and a 50 Hz servo pulse. The task functions separate terminal interaction, protocol exchange, and PWM conversion.